

Plot Memory - Dokumentation

Übersicht

Plot Memory ist eine neue, innovative Funktion für NeuroCore, die die traditionelle Chat-History durch intelligente, narrative Plot-Zusammenfassungen ersetzt. Dies reduziert den Token-Verbrauch drastisch und ermöglicht ein tieferes Kontext-Verständnis durch das LLM.

Motivation & Vorteile

Problem mit traditioneller Chat-History

- **Hoher Token-Verbrauch:** Vollständige Chat-History kann Tausende von Tokens verbrauchen
- **Redundante Informationen:** Viele Nachrichten enthalten wiederholte Informationen
- **Fehlende Struktur:** Einzelne Nachrichten ohne narrativen Zusammenhang
- **Context Window Limits:** Begrenzte Kapazität für lange Konversationen

Lösung: Plot Memory

Plot Memory ersetzt die detaillierte Chat-History durch eine **intelligente, narrative Zusammenfassung** die:

1.  **Token-Verbrauch reduziert** (oft 50-70% Reduktion)
 2.  **Kontext verbessert** durch kohärente Story-Struktur
 3.  **Alte Erinnerungen einbezieht** basierend auf aktueller Situation
 4.  **Intelligente Filterung** von relevanten Informationen
 5.  **Emotionalen Verlauf** und Charakterentwicklung berücksichtigt
-

Wie funktioniert Plot Memory?

Workflow



Intelligente Erinnerungsauswahl

1. Relevanz-basiert:

- Keyword-Match mit aktueller Nachricht
- Emotionale Valenz (intensive Momente)
- Retrieval-Count (oft abgerufene Erinnerungen)
- Konsolidierungsstatus

2. Zeitspanne:

- Configurable "Wie weit zurück schauen" (default: 100 Nachrichten)
- Alte signifikante Erinnerungen werden zusätzlich einbezogen

3. Spreading Activation:

- Semantisches Wissen wird über Knoten-Beziehungen aktiviert
- Indirekt relevante Konzepte werden einbezogen

Verwendung

Aktivierung

1. Öffne **NeuroCore Settings** in SillyTavern
2. Navigiere zum Tab **"Einstellungen"**
3. Scrolle zu **"Plot Memory"** Sektion
4. Aktiviere den Toggle **"Plot Memory aktivieren"**
5. Konfiguriere optionale Parameter (siehe unten)

Einstellungen

Allgemeine Einstellungen (gelten für beide Modi)

- **Max Slots (Working Memory):** 1-20 (Default: 8)
- Anzahl der Memory-Slots im Standard-Modus
- **Token Budget (%):** 1-50% (Default: 15%)
- Maximaler Prozentsatz des Context Window für Memory
- **Konsolidierungs-Intervall:** 5-100 (Default: 10)
- Nach wie vielen Nachrichten Konsolidierung erfolgt

Plot Memory Optionen (nur wenn aktiviert)

- **Max Episoden für Plot:** 5-50 (Default: 20)
- Wie viele Episoden maximal für Plot-Generierung verwendet werden
- **Max Semantische Knoten:** 5-30 (Default: 15)
- Wie viele Wissensknoten einbezogen werden
- **Zeitspanne (Nachrichten zurück):** 20-500 (Default: 100)
- Wie weit in die Vergangenheit geschaut wird
- **Emotionalen Verlauf einbeziehen:** ✓/✗ (Default: ✓)
- Ob emotionale Entwicklung im Plot berücksichtigt wird

Technische Details

Architektur

```

NeuroController
├── PlotGenerator (neu)
│   ├── _gatherMemoryData()
│   │   ├── Hippocampus.recall()
│   │   ├── _findOldSignificantMemories()
│   │   ├── _gatherSemanticKnowledge()
│   │   ├── Cerebellum.matchPatterns()
│   │   └── _extractEmotionalThemes()
│   ├── _analyzeStructure()
│   │   ├── _buildTimeline()
│   │   ├── Character Analysis
│   │   ├── Location Mapping
│   │   └── _clusterByTheme()
│   ├── _generatePlotWithLLM()
│   │   ├── _buildPlotGenerationPrompt()
│   │   └── llmCallback()
└── processMessage()
    └── (aktiviert Plot-Generierung wenn enablePlotMemory=true)
  
```

Dateien

Neue Dateien:

- `core/PlotGenerator.js` - Hauptlogik für Plot-Generierung

Modifizierte Dateien:

- `core/NeuroController.js` - Integration von PlotGenerator
- `index.js` - UI Event Handling
- `settings.html` - UI für Plot Memory Einstellungen

API

PlotGenerator Klasse

```
class PlotGenerator {
  constructor({ db, hippocampus, temporalLobe, cerebellum,
               basalGanglia, amygdala, spreadingActivation,
               llmCallback })

  // Generiert Plot aus aktuellen Erinnerungen
  async generatePlot(queryKeywords, currentMessageCount, options)

  // Cache Management
  isCacheValid()
  getCachedPlot()
  invalidateCache()
}
```

NeuroController Erweiterungen

```
class NeuroController {
  // Neue Properties
  this.plotGenerator      // PlotGenerator Instanz

  // Neue Settings
  settings.enablePlotMemory      // Boolean
  settings.plotMemoryOptions = {
    maxEpisodes,
    maxSemanticNodes,
    maxPatterns,
    includeEmotionalArc,
    timeSpanMessages
  }

  // Neue Methoden
  async rebuildInjectionAsync()      // Async rebuild mit Plot
  _buildStandardInjection()          // Standard-Modus helper
}
```

Best Practices

Wann Plot Memory nutzen?

✓ Empfohlen für:

- Lange Konversationen (>50 Nachrichten)
- Story-basierte Chats mit Charakterentwicklung

- Szenarien mit begrenztem Context Window
- Wenn Token-Kosten ein Faktor sind

✗ Weniger geeignet für:

- Kurze Chats (<20 Nachrichten)
- Rein technische/faktische Konversationen ohne Narrative
- Wenn jedes Detail der History wichtig ist
- Wenn LLM-Calls für Plot-Generierung zu langsam sind

Performance-Optimierung

1. **Cache nutzen:** Plot wird für 30 Sekunden gecached
2. **Konsolidierung:** Regelmäßige Konsolidierung verbessert Plot-Qualität
3. **Zeitspanne anpassen:** Kleinere Zeitspanne = schnellere Generierung
4. **Max Episoden:** Weniger Episoden = schnellere Verarbeitung

Fehlerbehebung

Problem: Plot ist zu kurz/oberflächlich

- Lösung: Erhöhe "Max Episoden" und "Max Semantische Knoten"
- Stelle sicher, dass genug Erinnerungen vorhanden sind (>20)

Problem: Plot-Generierung dauert zu lange

- Lösung: Reduziere "Zeitspanne" und "Max Episoden"
- Prüfe LLM-Verbindung und Geschwindigkeit

Problem: Plot enthält irrelevante Informationen

- Lösung: Reduziere "Zeitspanne"
- Stelle sicher, dass Keywords gut extrahiert werden

Problem: Alte wichtige Erinnerungen fehlen

- Lösung: Erhöhe "Zeitspanne (Nachrichten zurück)"
- Stelle sicher, dass emotionale Valenz korrekt gesetzt ist

Vergleich: Standard vs Plot Memory

Aspekt	Standard Memory	Plot Memory
Token-Verbrauch	Hoch (100% der Slots)	Niedrig (50-70% Reduktion)
Struktur	Liste von Einzelerinnerungen	Kohärente Narrative
Alte Erinnerungen	Nur via Retrieval	Intelligent einbezogen
Kontext-Qualität	Fragmentiert	Zusammenhängend
Verarbeitungszeit	Schnell (kein LLM)	Mittel (LLM-Call nötig)
Cache	Statisch	30s Cache
Anpassbarkeit	Begrenzt (nur Slots)	Viele Optionen

Beispiele

Beispiel 1: Standard Memory Output

```
[NeuroCore Memory Injection – START]

## Aktive Erinnerungen (Working Memory):
- User fragt nach dem Wetter heute
- Character sagt es wird sonnig
- User erwähnt den Park
- Character schlägt einen Spaziergang vor
- User stimmt zu
- Character freut sich
- ...

## Bekannte Charaktere:
- Emma: Freundin, mag Natur

[NeuroCore Memory Injection – ENDE]
```

Token Count: ~450 Tokens

Beispiel 2: Plot Memory Output

```
[NeuroCore Plot Memory – START]

Die Konversation entwickelt sich um eine freundschaftliche Beziehung zwischen User und Emma. Emma ist eine naturverbundene Person, die gerne Zeit im Freien verbringt.

In den letzten Interaktionen wurde ein Spaziergang im Park geplant. Das Wetter wurde als sonnig beschrieben, was beide als positiv bewerteten. Emma zeigte besondere Begeisterung für diese Aktivität.

Charaktere: Emma (Freundin, naturverbunden, enthusiastisch)
Wichtige Orte: Park
Emotionale Stimmung: Positiv, vorfreudig

[NeuroCore Plot Memory – ENDE]
```

Token Count: ~180 Tokens (60% Reduktion!)

Zukunft & Erweiterungen

Geplante Features

- [] **Mehrere Plot-Stile:** Dramatisch, Sachlich, Poetisch
- [] **Plot-Versioning:** Automatisches Tracking von Plot-Änderungen
- [] **User-Feedback:** Manuelles Korrigieren von Plots
- [] **Plot-Visualisierung:** Grafische Darstellung der Story-Struktur
- [] **Export/Import:** Plots als separate Dateien speichern
- [] **Multi-Character Support:** Bessere Handhabung mehrerer Charaktere

Experimentelle Optionen

Diese Features sind in der aktuellen Version noch nicht implementiert, könnten aber in Zukunft hinzugefügt werden:

- **Adaptive Zeitspanne:** Automatische Anpassung basierend auf Konversationsdichte
 - **Importance Scoring:** ML-basierte Bewertung der Wichtigkeit von Erinnerungen
 - **Plot Branching:** Multiple alternative Story-Stränge
 - **Real-time Updates:** Live-Aktualisierung des Plots während des Chats
-

FAQ

Q: Wird mein Chat gelöscht wenn ich Plot Memory aktiviere?

A: Nein! Alle Erinnerungen bleiben in der Datenbank erhalten. Plot Memory ändert nur, wie die Informationen dem LLM präsentiert werden.

Q: Kann ich zwischen Standard und Plot Memory wechseln?

A: Ja, jederzeit! Der Modus kann in den Einstellungen umgeschaltet werden.

Q: Brauche ich einen speziellen LLM für Plot Memory?

A: Nein, es funktioniert mit allen LLMs die NeuroCore unterstützt.
Bessere LLMs (GPT-4, Claude) generieren qualitativ hochwertigere Plots.

Q: Was passiert wenn die Plot-Generierung fehlschlägt?

A: Das System fällt automatisch auf den Standard-Modus zurück (Fallback).

Q: Wie viele Tokens spart Plot Memory durchschnittlich?

A: Im Durchschnitt 50-70%, kann aber je nach Konversation variieren.

Q: Beeinflusst Plot Memory die Qualität der LLM-Antworten?

A: Ja, positiv! Durch die narrative Struktur versteht das LLM den Kontext oft besser als durch fragmentierte Einzelerinnerungen.

Support & Feedback

Bei Fragen oder Problemen:

1. Prüfe die Konsole auf Fehlermeldungen (`[PlotGenerator]` oder `[NeuroCore]`)
 2. Stelle sicher, dass die LLM-Verbindung funktioniert
 3. Teste mit deaktivierten Plot Memory ob das Problem weiterhin besteht
 4. Erstelle ein Issue auf GitHub mit Details zur Konfiguration
-

Version: 1.0.0

Datum: März 2026

Author: NeuroCore Team